הרצאה 8: אלגוריתמים חמדניים

בהרצאה זו נציג את מושג האלגוריתם החמדני (Greedy Algorithm). לאחר מכן נציג את הבעיה החישובית **תזמון משימות למשאב יחיד** ונפתור את הבעיה באמצעות אלגוריתם חמדני. בחלקה השני של ההרצאה נציג את הבעיה החישובית **חישוב קוד בינארי אופטימלי**. בהרצאה הבאה נציג את האלגוריתם החמדני של הפמן (Huffman) לחישוב קוד בינארי אופטימלי לקובץ קלט נתון.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

יסודות השיטה החמדנית

אלגוריתם חמדני בוחר תמיד את צעדו הבא באמצעות **כלל בחירה** אשר מביא לו תועלת מרבית מיידית, ללא התחשבות בהשלכות הבחירה על המשך הפתרון. הכינוי ״אלגוריתם חמדני״ ממשיל את האלגוריתם לאדם חמדני אשר בהינתן בידיו הזדמנות להיטיב את מצבו הוא מנצל אותה ללא היסוס וללא התחשבות בהשלכותיה לטווח הארוך.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

דוגמה שלילית

לדוגמה נתבונן בבעיה *Sum* שכבר פתרנו באמצעות תכנות דינמי:

הקלט: מערך מספרים חיוביים, A.

הפלט: סכום מרבי של איברי תת-מערך של A אשר אינו מכיל שני איברים סמוכים זה לזה.

איור 8.1: הבעיה החישובית *Sum*

אלגוריתם *GreedySum* שנציג כעת הוא דוגמה לאלגוריתם חמדני אשר **נכשל** בפתרון הבעיה *Sum*.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

כלל הבחירה

כלל הבחירה של אלגוריתם *GreedySum* הוא: **בכל שלב לצרף לסכום את האיבר המרבי שטרם צורף ושהוא גם אינו שכן לאיבר שכבר נבחר**. הקוראות והקוראים זריזי המחשבה ישימו לב מייד שאלגוריתם *GreedySum* המופעל עם הקלט $[5,7,4]$ מחזיר את הפלט 7, כאשר הפלט הנכון הוא כמובן 9. במקרה זה כלל הבחירה המוצע אינו מוביל לפתרון הבעיה (כפי שראינו בפרק הקודם, אפשר לפתור את הבעיה באמצעות תכנון דינמי).

© כל הזכויות שמורות לפרופסור עמוס ישראלי

כדוגמה לבעיה חישובית הניתנת לפתרון באמצעות אלגוריתם חמדני פשוט, נציג כעת את הבעיה **תזמון משימות למשאב יחיד**.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגדרה 8.2: משאב ומשימות

1. **משאב (resource)** הוא עצם הדרוש לביצוע משימות מסוימות. משאב יכול להיות כיתת לימוד, שירות תוכנה, מחשב נייד מחלקתי וכדומה. בכל רגע נתון המשאב יכול לספק שירות למשימה אחת לכל היותר.
2. **משימה (task)** היא דרישת שירות ממשאב כלשהו. כל מטלה, t, מאופיינת על ידי פרק הזמן שבו היא צריכה להתבצע, הקרוי **מרווח הפעולה** של t. מרווח הפעולה של t מסומן על ידי הקטע הפתוח $(t.start, t.finish)$.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגדרה 8.3: תזמון

1. נתון מערך משימות *Tasks*. **תִזמון (schedule)** של המערך *Tasks* הוא תת-מערך *Sched* אשר מרווחי הפעולה של המשימות הנכללות בו הם זרים וממוינים בסדר עולה של זמני ההתחלה שלהם (וגם של זמני הסיום). **זמן הסיום** של התזמון *Sched* הוא זמן הסיום של המשימה האחרונה ב-*Sched*.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

2. תזמון *Sched* של מערך משימות *Tasks* הוא **אופטימלי** אם הוא מקיים:

2.1 מספר המשימות בכל תזמון של *Tasks* קטן ממספר המשימות ב-*Sched* או שווה לו.

2.2 זמן הסיום של כל תזמון של *Tasks* אשר מספר איבריו שווה למספר איברי *Sched* גדול מזמן הסיום של *Sched* או שווה לו.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הבעיה החישובית: תזמון משימות (task scheduling) למשאב יחיד

הקלט:

מערך משימות *Tasks* אשר מקיים: זמני ההתחלה וזמני הסיום של כל המשימות ב-*Tasks* שונים זה מזה.

הפלט:

תזמון אופטימלי של *Tasks*.

איור 8.4: תזמון משימות למשאב יחיד

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הכנות לאלגוריתם חמדני

קיימות אפשרויות לא מעטות להגדרת כלל בחירה לאלגוריתם חמדני לפתרון בעיה זו. בתרגיל 8.1 מוצעים שלושה כללי בחירה, ואתם מתבקשים להראות שכל אחד מכללי הבחירה המוצעים אינו מניב תזמון אופטימלי.

אלגוריתם *GSched* שנציג כעת מקבל כקלט מערך משימות *Tasks* ומחזיר כפלט את *OutSched*, תזמון אופטימלי של *Tasks*. כלל הבחירה של אלגוריתם *GSched* מסתמך על ההגדרה הזו:

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגדרה 8.5

יהי T מערך משימות ותהי t משימה אשר אינה כלולה ב-T. המשימה t **מותרת ל-T** אם מרווח הפעולה של t זר למרווחי הפעולה של כל אחת מן המשימות ב-T.

כלל הבחירה של אלגוריתם $GSched$ הוא: **בכל שלב צרף ל-$OutSched$ את המשימה המותרת שזמן הסיום שלה הוא המינימלי מבין המשימות המותרות בתחילת אותו שלב.**

© כל הזכויות שמורות לפרופסור עמוס ישראלי

מבני הנתונים

מבני הנתונים אשר אלגוריתם *GSched* משתמש בהם הם:

- מערך משימות בן n איברים *STasks* אשר מאחסן את משימות מערך הקלט לאחר מיונן בסדר עולה של זמני הסיום שלהן.
- מערך משימות *OutSched* אשר מאחסן את תזמון הפלט.
- משתנה *endtime* המאחסן את זמן הסיום הנוכחי של תזמון הפלט אשר נבנה במערך *OutSched*.
- משתנה *nextInd* המאחסן מצביע אל האיבר במערך הפלט *OutSched* שבו תאוחסן המשימה המצורפת הבאה.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

אלגוריתם *GSched*

```
GSched(Tasks)
    STasks ⇐ Sort(Tasks)
    OutSched[1] ← STasks[1]
    endtime ← OutSched[1].finish
    nextInd ← 2
    for i ← 2 to n
        if (STasks[i].start ≥ endtime)
            OutSched[nextInd] ← STasks[i]
            nextInd ← nextInd +1
            endtime ← STasks[i].finish
        end if
    end for
    return(OutSched)
end(GSched)
```

איור 8.6: אלגוריתם *GSched*

© כל הזכויות שמורות לפרופסור עמוס ישראלי

תיאור האלגוריתם

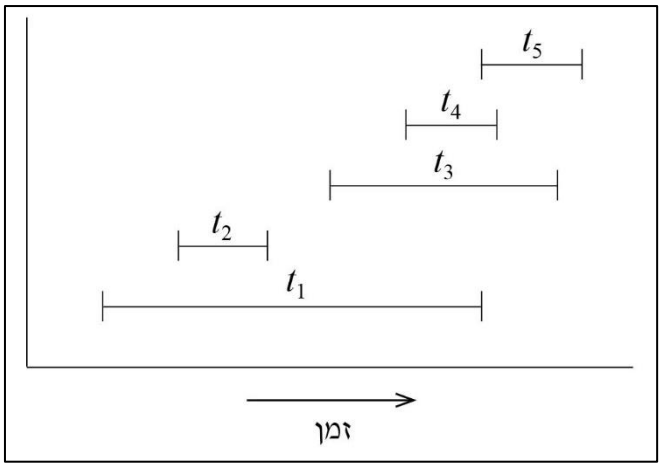
האלגוריתם נפתח במיון המשימות המאוחסנות במערך הקלט *Tasks* בסדר עולה של זמני הסיום שלהן. בדרך זו איתור המשימה המועמדת לצרוף ל-*OutSched* בכל שלב מתבצע בסיבוכיות קבועה. לאחר המיון, המשימה שזמן הסיום שלה מינימלי מצורפת לתזמון הפלט והאלגוריתם מאתחל את המשתנים *endtime* ו-*nextInd*.

לאחר מכן, האלגוריתם ממשיך בבניית *OutSched* באמצעות לולאה. במעבר ה-i-י בלולאה, $i = 2,\ldots,n$,

© כל הזכויות שמורות לפרופסור עמוס ישראלי

האלגוריתם בודק האם המשימה $STasks[i]$ מותרת למשימות שכבר מאוחסנות במערך $OutSched$, כלומר האם $STasks[i].start > endtime$.
אם התשובה חיובית, האלגוריתם מאחסן את המשימה $STasks[i]$ באיבר $OutSched[nextInd]$, המשתנה $nextInd$ מקודם ב-1 והמשתנה $endtime$ מעודכן לזמן הסיום של המשימה $STasks[i]$. אם התשובה שלילית, המעבר ה-i-י מסתיים ללא שינוי במשתני האלגוריתם.

© כל הזכויות שמורות לפרופסור עמוס ישראלי



איור 8.7: קלט לאלגוריתם *GSched*

באיור 8.7 מוצג קלט אופייני *GSched*.
באיור זה מוצגים מרווחי הפעולה של
המשימות $t_1,\ldots,t_5$. אפשר לראות
שהמשימה הראשונה שתצורף
ל-*OutSched* היא t_2, ומשימה זו
"פוסלת" את המשימה t_1. לאחר מכן
המשימה t_4 מצורפת ל-*OutSched*
ומשימה זו "פוסלת" את t_3 ואת t_5.
בכך מסתיים חישוב תזמון הפלט.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

מעקב פשוט אחר ביצוע *GSched* מראה שבכל שלב של האלגוריתם המערך *OutSched* הוא תזמון חוקי. כדי להשלים את הוכחת הנכונות, עלינו להראות ש-*OutSched* הוא גם תזמון אופטימלי.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

משפט 8.8

יהי $Tasks$ מערך קלט לאלגוריתם $GSched$ שמספר איבריו הוא n. נניח שבכל תזמון אופטימלי למשימות ב-$Tasks$ יש l משימות. הסדרה $Outsched[1],\ldots,Outsched[l]$ היא תזמון אופטימלי של $Tasks$.

הוכחה

יהי $t_1,\ldots,t_l$ תזמון אופטימלי כלשהו של המשימות במערך הקלט $Tasks$.

להלן נוכיח באינדוקציה על i שהתזמון $Outsched[1],\ldots,Outsched[i]$ הוא רישה של תזמון אופטימלי.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

<u>בסיס</u> $(i=1)$

אם $t_1 = OutSched[1]$ נכונות הטענה ברורה. נניח אם כן ש-$t_1 \neq OutSched[1]$. נזכור שהמשימות המאוחסנות במערך $OutSched$ ממוינות לפי סדר עולה של זמני הסיום שלהן, כלומר:

$$Outsched[1].finish < t_1.finish < t_2.finish < \cdots < t_l.finish$$

מכאן נובע שסדרת המשימות $OutSched[1], t_2, \ldots, t_l$ המתקבלת מהחלפת המשימה t_1 במשימה $OutSched[1]$ היא תזמון אופטימלי והרישה שלו היא $OutSched[1]$.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

צעד

נניח שסדרת המשימות

$OutSched[1],\ldots,OutShed[i]$ היא רישה

של תזמון אופטימלי

$OutSched[1],\ldots,OutSched[i],t_{i+1},\ldots,t_l$.

אם $t_{i+1} = Outsched[i+1]$, נכונות

הטענה ברורה. נניח אם כן

ש-$t_{i+1} \neq OutSched[i+1]$.

הוכחת הטענה במקרה זה זהה הוכחת

הטענה במקרה הבסיס. **מש"ל**

© כל הזכויות שמורות לפרופסור עמוס ישראלי

מסקנה 8.9

אלגוריתם *GSched* המקבל כקלט מערך משימות *Tasks* מחזיר תזמון אופטימלי *OutSched* של המשימות במערך *Tasks*.

הוכחה

הוכחת הטענה נובעת מיידית מהנחת האינדוקציה עבור $i = l$.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

סיבוכיות אלגוריתם *GSched*

מיון מערך הקלט *Tasks* ניתן לביצוע בסיבוכיות $\Theta(n\log n)$. הסיבוכיות של כל אחת מן ההוראות המתבצעות בכל מעבר בלולאת האלגוריתם היא קבועה. מכאן נובע שסיבוכיות אלגוריתם *GreedySched* היא $\Theta(n\log n)$.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

חישוב קוד בינארי אופטימלי

נמשיך את ההרצאה בהצגת הבעיה החישובית **חישוב קוד בינארי אופטימלי**. בהרצאה הבאה נציג את האלגוריתם החמדני של הפמן (Huffman) הפותר בעיה זו.

נניח שברצוננו לאחסן קובץ טקסט F בזיכרון המחשב. הקובץ F הוא סדרת תווים מעל אלף-בית בית כלשהו. כדוגמה, נזכיר את האלף-בית ASCII המכיל 128 תווים שונים. מאחר שה-״אלף-בית״ המחשבי הוא בינארי, נחוצה דרך לקידוד תווי האלף-בית של F באמצעות סדרה בינארית (סדרה שאיבריה הם 0-ים ו-1-ים.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגדרה 8.10: אלף-בית וקובץ

1. **אלף-בית**, $C=\{c_1,\ldots,c_n\}$ הוא קבוצה סופית של איברים המכונים **תווים** (characters).
2. **קובץ** F מעל אלף-בית C הוא סדרת תווים, $F=c_1,c_2,\ldots,c_l$ כאשר כל אחד מן התווים ב-F הוא איבר באלף בית C.

נניח כי אנו רוצים לאכסן בזיכרון המחשב קובץ כלשהו מעל C.
כדי לאחסן קובץ בזיכרון המחשב נקודד כל תו על ידי מחרוזת בינארית.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגדרה 8.11: קוד בינארי

(Binary Coding)

נתון אלף-בית, $C=\{c_1,c_2,...,c_n\}$. **קוד בינארי** (להלן **קוד**) של C, הוא פונקציה, k, המתאימה לכל תו $c\in C$, מחרוזת סיביות $k(c)$.

המחרוזת $k(c)$ נקראת **הקוד** של c.

מספר הסיביות ב-$k(c)$, מסומן ב-$|k(c)|$.

לדוגמה

נתבונן באלף בית $C=\{a,b,c\}$ ונגדיר: $k(a)=0$, $k(b)=10$ ו-$k(c)=11$.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגדרה 8.12: קוד של קובץ

יהי $F = c_1,\ldots,c_l$ קובץ מעל אלף בית C ויהי k קוד של C. **הקוד** של F, $k(F)$, הוא שרשור של הקודים של התווים ב-F.

$$k(F) = k(c_1) \circ \ldots \circ k(c_l)$$

נסמן ב-$|k(F)|$ את מספר התווים ב-$k(F)$.

להלן F יכונה **קובץ המקור**,

ו-$k(F)$ יכונה **הקובץ המקודד**.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

דוגמא לקוד בינארי ולקוד של קובץ

באיור 8.13 מוצגת דוגמה לקוד בינארי לאלף-בית $C=\{a,b,c,d,e,f\}$.

תו c	f	e	d	c	b	a
קוד $k_1(c)$	101	110	011	010	001	000

איור 8.13: דוגמה לקוד בינארי

לדוגמה

הקוד של הקובץ adf הוא

$$k_1(adf)=000\circ 011\circ 101=$$

$$=000011101$$

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגדרה 8.14: קוד חד-פענח

הדרישה הבסיסית מקוד כלשהו היא שאם F_1 ו-F_2 הם שני קבצים **שונים זה מזה** מעל אלף בית C ואם k הוא קוד ל-C, אזי הקוד של F_1 **שונה** מהקוד של F_2. כלומר $k(F_1) \neq k(F_2)$.

קוד המקיים דרישה זו הוא

קוד חד-פענח

(Uniquely Decipherable Code)

© כל הזכויות שמורות לפרופסור עמוס ישראלי

דוגמה לקוד שאינו חד פענח

$$k(a)=0,\ k(b)=1,\ k(c)=10$$

$$k(aba)=k(ac)=010$$

כלומר לקובץ המקודד 010 יש שני קבצי מקור.

כדי להבטיח שחזור חד משמעי של קובץ מקודד נדרש שכל קוד בינארי יהיה חד-פענח.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

השוואה בין שני קודים

נניח שנתונים לנו שני קודים לאלף בית C, k_1 ו-k_2. נניח שכל אחד מן הקודים הוא חד פענח. איך נחליט באיזה קוד להשתמש? בלים אחרות:

איזה קוד טוב יותר?

תשובה

לכל קובץ F ככל שמספר התווים בקובץ המקודד, $K(F)$, קטן יותר, הקוד טוב יותר.

מסקנה חשובה

לכל קובץ מעל אלף בית C. מתאים קוד שונה.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגדרה 8.15: קוד שווה אורך וקוד באורך משתנה

- קוד k הוא **שווה אורך** (**Fixed-length Code word**), אם לכל שני תווים, c_i, ו-c_j מתקיים $|k(c_i)|=|k(c_j)|$.
- קוד שאינו שווה-אורך, הוא **קוד באורך משתנה** (**Variable-length Code word**).

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגדרה 8.16: שכיחות של תו בקובץ

יהי F קובץ כלשהו מעל אלף בית C ויהי c תו כלשהו ב-C. **השכיחות** של c בקובץ F היא מספר המופעים של c ב-F . נסמן את השיחות של c ב-F באמצעות $f\left(c:F\right)$.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

השוואה בין קוד שווה אורך וקוד באורך משתנה

נניח כי ברצוננו לקודד קובץ F, ובו 100 מלים מעל א״ב בן 6 תווים, ונניח גם כי לכל תו ב-F יש שכיחות שונה.

קוד יעיל של F מתאים מחרוזות בינאריות קצרות לתווים בעלי שכיחות גבוהה ומחרוזות ארוכות יותר לתווים ששכיחותם נמוכה יותר.

באיור 8.17 מוצגת טבלת השוואה בין שני קודים לקובץ F. בשורה הראשונה באיור זה מוצגים תווי האלף בית C.

בשורה השנייה מוצגת השכיחות של כל אחד מן התווים ב-F, $f\left(c,F\right)$. בשורה

© כל הזכויות שמורות לפרופסור עמוס ישראלי

השלישית מוצג **קוד שווה אורך**, k_1.
בשורה הרביעית מופיע **קוד באורך משתנה**, k_2.

תו	f	e	d	c	b	a
שכיחות	5	9	16	12	13	45
k_1-שווה אורך	101	110	011	010	001	000
k_2-אורך משתנה	1100	1101	111	100	101	0

<u>איור 8.17: טבלת שכיחויות וקודים</u>

קל לראות כי $|k_1(F)|=300$
ו-$|k_2(F)|=222$.

דוגמה זו מבהירה מייד כי קוד באורך משתנה, המתחשב בשכיחות כל תו בקובץ המקור, יעיל יותר מקוד שווה אורך. מטרתנו היא לפתח אלגוריתם יעיל המקבל כקלט קובץ תווים ומחזיר קוד בינארי אופטימלי כפי שנגדיר.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגדרה 8.18: קוד אופטימלי לקובץ

נתון קובץ F מעל אלף-בית C. קוד k מעל C, הוא **אופטימלי** עבור F אם לכל קוד אחר מעל C, k', מספר הסיביות ב-$k(c)$ קטן ממספר הסיביות ב-$k'(c)$ או שווה לו.

אם k הוא קוד אופטימלי עבור F נאמר שהקוד k הוא **אופטימלי עבור הזוג** $\langle C, F \rangle$.

שימו לב:

לכל קובץ מתאים קוד אופטימלי משלו.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

מסענו להצגת אלגוריתם לחישוב קוד בינארי אופטימלי ממשיך לפי השלבים שלהלן:

1. נגדיר סוג של קוד בינארי הקרוי **קוד תחיליות**.
2. נציג (אך לא נוכיח) את משפט קראפט (Kraft) ונשתמש במשפט כדי להוכיח שלכל קובץ F מעל אלף-בית C קיים קוד תחיליות k אשר הוא אופטימלי ביחס לזוג $\langle C,F\rangle$.
3. נגדיר מהו עץ קוד ונראה שניתן לייצג כל קוד תחיליות, k, באמצעות עץ קוד שנסמנו ב- T_k.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

4. נציג את אלגוריתם הפמן (Huffman) המקבל כקלט קובץ תווים F מעל אלף-בית C ומחזיר עץ קוד המייצג קוד אופטימלי עבור הזוג $\langle C, F \rangle$.

5. נוכיח את נכונות אלגוריתם הפמן וננתח את הסיבוכיות שלו.

<u>הגדרה 8.19: קוד תחיליות (Prefix Code)</u>

קוד תחיליות הוא קוד בינארי המקיים:

לכל שני תווים c_1 ו-c_2, הקוד של c_1 אינו תחילית (התחלה, רישה) (באנגלית prefix) של הקוד של c_2.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

דוגמאות

1. הקוד המוגדר על ידי:

$$K(a)=0$$

$$K(b)=01$$

אינו קוד תחיליות מפני שהקוד של התו a הוא תחילית של הקוד של b.

2. בקוד שווה אורך, הקודים של כל התווים הם באורך זהה, ולכן לא ייתכן מצב שבו קוד של תו מסוים הוא תחילית של קוד של תו אחר.
מכאן: כל קוד שווה אורך הוא קוד תחיליות.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הבחנה 8.20

כל קוד תחיליות הוא חד פענח.

הוכחה

ההוכחה נותרת כתרגיל לסטודנטיות המצטיינות.

השם קוד תחיליות הוא מטעה, השם הזה ניתן מסיבות היסטוריות. שם מתאים יותר הוא **קוד חסר תחיליות**.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

משפט 8.21: משפט קראפט (Kraft)

משפט Kraft מוכיח כי לכל קובץ תווים F מעל אלף-בית C ולכל קוד k, אם k אינו קוד תחיליות, אזי קיים קוד תחיליות k' כך שמתקיים

$$|k'(F)|=|k(F)|$$

הוכחת משפט קראפט היא מעבר לתחום ההרצאה.

מסקנה

לכל זוג $\langle C,F\rangle$ קיים **קוד תחיליות** k אופטימלי עבור $\langle C,F\rangle$.

הוכחת המסקנה

נותרת כתרגיל לסטודנטים.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

ייצוגי קוד

כדי לקודד קובץ תווים עלינו לטפל ב:

1. **קידוד** קובץ המקור.
2. **אחזור (פיענוח)** הקובץ המקודד.

ייצוג קוד באמצעות מערך (טבלה)

מתאים מאוד לקידוד: האלגוריתם עובר על פני איברי קובץ המקור ועבור כל איבר, a, האלגוריתם ניגש למערך המייצג, ומעתיק את $k(a)$ לקובץ המקודד. **פיענוח** קובץ מקודד באמצעות טבלה אינו פשוט. כדי לפענח קובץ מקודד נייצג את קוד התחיליות באמצעות **עץ קוד** שנגדיר כעת.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

ייצוג קוד תחיליות באמצעות עץ קוד

לכל קוד תחיליות, k, נייצג את k

באמצעות **עץ קוד המתאים ל-**k .

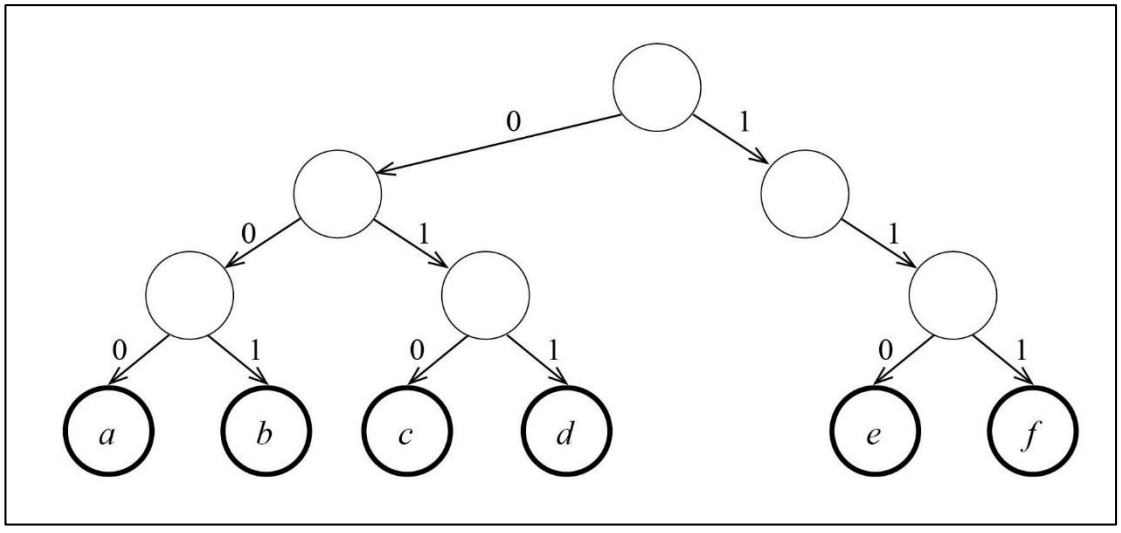
דוגמה

באיור 8.22 מוצג הקוד k_1. באיור 8.23

מוצג עץ הקוד המתאים ל-k_1.

התו	f	e	d	c	b	a
קוד k_1	111	110	011	010	001	000

איור 8.22: הקוד k_1



איור 8.23: עץ הקוד T_{k_1}

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגדרה 8.24: עץ קוד (הגדרה פורמלית)

יהי k קוד תחיליות של אלף-בית, C, $C=\{c_1,c_2,...,c_n\}$. **עץ הקוד המייצג את k**, $T_k=(V_k,E_k)$, הוא עץ בינארי מתויג, כאשר לכל צומת פנימי, $v\in V_k$, אחת הקשתות היוצאת מ-v מתויגת על ידי 0 והקשת השניה מתויגת על ידי 1. (אם מ-v יוצאת רק קשת אחת, היא מתויגת ב-0, או ב-1). לכל תו ב-C מתאים ענף (מסלול משורש העץ לאחד העלים) יחיד ב-T_k. נסמן את הענף המתאים לתו c_i, ב-α_i. העלה בקצהו של α_i מתויג על ידי c_i והקשתות

© כל הזכויות שמורות לפרופסור עמוס ישראלי

לאורך α_i, החל בשורש וכלה בעלה מתויגות על ידי סיביות הקוד $k(c_i)$.

במלים אחרות: אם $k(c_i) = b_1,\ldots,b_l$, אזי ב-α_i יש l קשתות, והקשת ה-j, $j = 1,\ldots,l$ (הספירה מן השורש ועד לעלה המתאים ל-c_i) מתויגת על ידי b_j.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

<u>פענוח קובץ מקודד באמצעות עץ קוד</u>

יהי F קובץ מקודד על ידי קוד תחיליות k. עץ הקוד של k, T_k, מאפשר אחזור (פענוח) של קובץ המקור בדרך הבאה: תווי קובץ המקור מאוחזרים בזה אחר זה כאשר כל אחד מתווי קובץ המקור מזוהה על ידי סיור משורש עץ הקוד T_k אל אחד העלים.

לדוגמה הקידוד של הקובץ daf הוא

$$k_1(daf) = 011 \circ 000 \circ 111 = 011000111$$

כדי לפענח את התו הראשון האלגוריתם מסייר במורד העץ החל מן השורש בהתאם לסיביות הקובץ המקודד.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הסיבית הראשונה בקובץ המקודד היא 0. בהתאם, בצעד הראשון בסיור עובר על פני הקשת היוצאת מן השורש ומתויגת ב-0, הצעד השני עובר על פני הקשת המקודדת ב-1 וכן הלאה. לאחר 3 צעדים הסיור מגיע אל העלה המתויג ב-d וזהו התו הראשון בקובץ מפוענח. פיענוח כל אחד מן התווים הנותרים מתבצע באופן דומה.

נמשיך בהגדרת העלות של עץ קוד ביחס לקובץ F

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגדרה 8.25: עלות של עץ קוד

נתון קובץ תווים F מעל אלף-בית $C=\{c_1,\ldots,c_n\}$. יהי k קוד בינארי של C ויהי T_k עץ הקוד המייצג את k. **עלות העץ T_k ביחס לקובץ F** מוגדרת כמספר הסיביות בקובץ המקודד, $k(F)$. נסמן ב- $f(i,F)$ את שכיחות התו c_i בקובץ F. מספר הסיביות בקובץ המקודד $k(F)$ שווה ל:

$$|k(F)|=\sum_{i=1}^{n}|k(i)|\cdot f(i).$$

נגדיר את עלות העץ T_k לקובץ F כמספר הסיביות ב-$K(F)$ ונסמן זאת ב- $Cost(T_k,F)$.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

נסמן ב-$l(i)$ את מספר הקשתות בענף המשתרע משורש T_k ועד הצומת המתויג ב-c_i ונזכור ש-$l(i)=|k(i)|$,

ונקבל:

$$Cost(F,T_k)=\sum_{i=1}^{n} l(i)\cdot f(i)$$

© כל הזכויות שמורות לפרופסור עמוס ישראלי

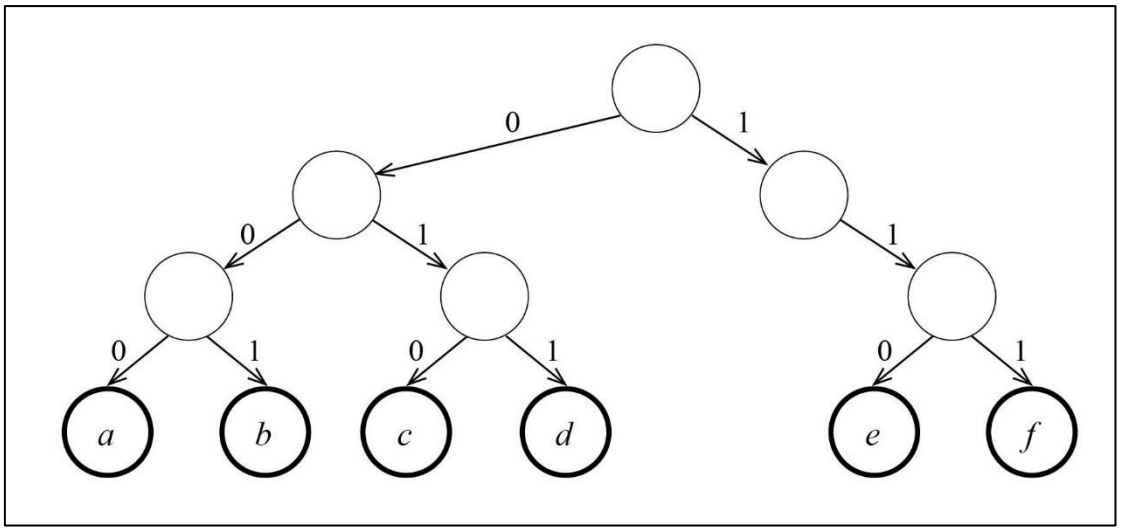
דוגמה

באיור 8.26 מופיע ייצוג טבלאי של k_2

תו	f	E	d	c	b	a
שכיחות	5	9	16	12	13	45
k_1-שווה אורך	101	110	011	010	001	000
k_2-אורך משתנה	1100	1101	111	100	101	0

איור 8.26

באיור 8.27 מוצג עץ הקוד המתאים ל-k_2.



איור 8.27: עץ הקוד של k_2.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

התו הראשון בקובץ הוא f אורך הקוד של f הוא 4 ובהתאם לכך, אורך המסלול משורש T_k אל העלה המתויג בתו f הוא 4.

התו f מופיע בקובץ פעמיים ולכן תרומתו הכוללת של התו f לאורך הקובץ המקודד היא 8 סיביות.

בדומה לכך, התו a תורם לקובץ המקודד סיבית אחת והתו b תורם 3 סיביות. מכאן נובע שעלות הקוד k_2 ביחס לקובץ F שווה ל-12.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגדרה 8.28: עלות של עץ קוד ביחס לקובץ F

יהי k קוד תחיליות של אלף-בית $C=\{c_1,c_2,...,c_n\}$ ויהי T_k עץ הקוד המייצג של C. לכל תו $c_i \in C$ נסמן ב-l_i את מספר הקשתות בענף המשתרע משורש העץ T_k ועד לצומת המתויג ב-c_i. לכל קובץ F מעל C נסמן ב-$f(i)$ את שכיחות התו c_i בקובץ F. **עלות** העץ T_k עבור הקובץ F מוגדרת כך:

$$Cost(T_k,F)=\sum_{i=1}^{n} l_i(i)\cdot f(i)=$$

$$\sum_{i=1}^{n} |k(i)|\cdot f(i)=|k(F)|$$

© כל הזכויות שמורות לפרופסור עמוס ישראלי

הגדרה 8.29: עץ קוד אופטימלי עבור קובץ F

עץ קוד T הוא **אופטימלי** עבור הזוג $\langle C, F \rangle$ אם לכל עץ קוד אחר ל-C, T', $Cost(T,F) \leq Cost(T',F)$.

לאחר העבודה הרבה שהשקענו עד הנה אנו יכולים להציג את הבעיה החישובית שנפתור בהרצאה הבאה.

© כל הזכויות שמורות לפרופסור עמוס ישראלי

חישוב קוד תחיליות אופטימלי

הבעיה החישובית **חישוב עץ קוד אופטימלי** מוצגת באיור 8.30.

הקלט:

קובץ תווים F מעל אלף בית C.

הפלט:

עץ קוד אופטימלי עבור הזוג $\langle C, F \rangle$

איור 8.30: חישוב עץ קוד אופטימלי

© כל הזכויות שמורות לפרופסור עמוס ישראלי